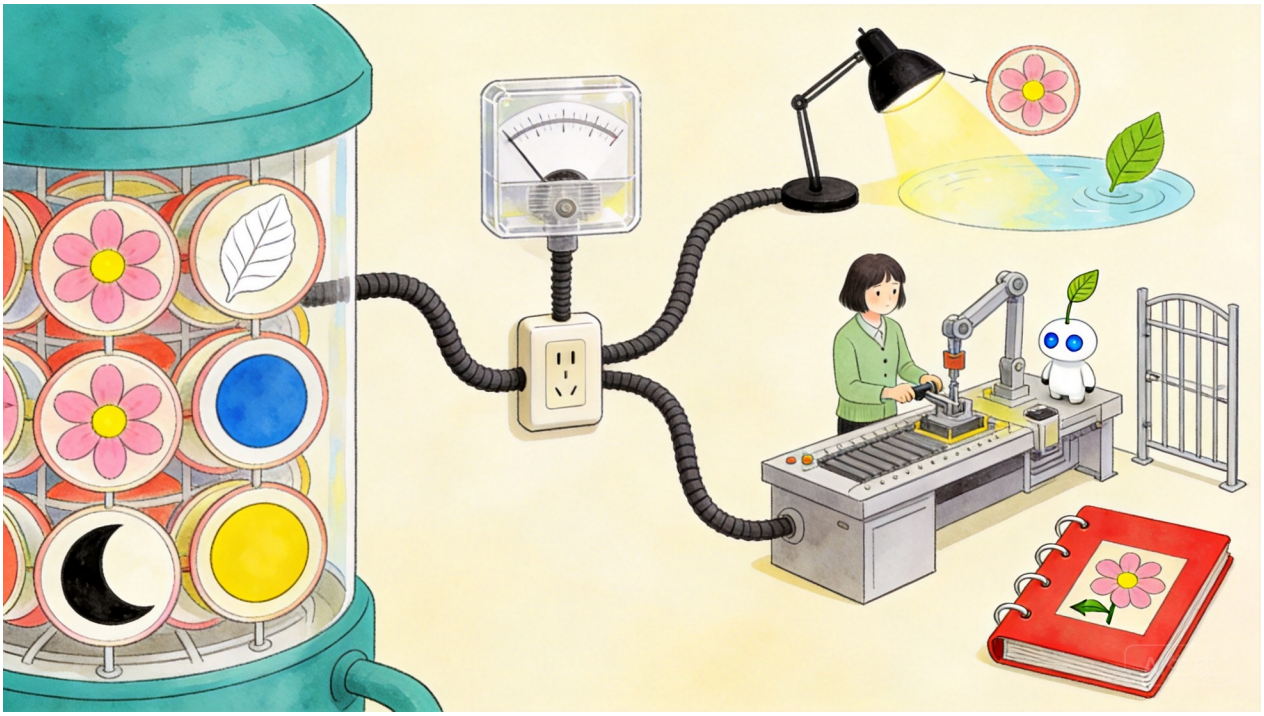
词元（Token）、大模型和智能体：三者究竟是什么关系？

“通过 ASKS 智能知识系统看 AI Agent”系列第 4 篇

ASKS（Agent-Driven Scientific Knowledge System）是我们的智能知识系统。

项目代码：github.com/ranshiju/Ran-ASKS

本文通过与 ASKS 系统交互完成。



云端大模型提供能力，Token 记录用量，Agent 把能力变成不同用途

我们使用冰箱、洗衣机和电脑，并不需要先在家里建一座发电厂。把插头接上电网，按用量交电费，就能让不同电器工作。

今天使用云端大模型，也有一点像这样。

多数人不需要购买大量计算设备、部署一个庞大的模型。应用通过 API 接入云端大模型，输入问题，得到输出，并按服务规则付费。在许多 API 中，用量正是按照输入和输出的 Token 数量计算的。

于是，一个很实用的比喻出现了：

- 云端大模型像智能电厂；
- API 像电网和插座；
- 模型提供的推理与语言能力像电力；
- Agent 像接入电网的电器，复杂 Agent 则更像一条自动化生产线；
- Token 像电表记录的用量。

这个比喻能把三个常被混在一起的概念分开。不过，它有一个必须立刻说明的边界：Token 并不是真的电，也不等于模型的“能量”。它首先是模型处理信息的基本单位，只是在许多云端服务中又被用作计费单位。



一句很短的任务背后，论文、Wiki 和规则都会形成需要计量的输入 Token

Token：电表究竟在数什么？

想象一座快递分拣站。整句话不能原封不动地送进大模型，而要先经过一台“切分机”：文字、数字、标点等内容被装成一个个带编号的小包裹，再按原来的顺序送上传送带。每个小包裹，就是一个 Token。

例如，“人工智能正在改变生活”在不同模型中，可能被切成几个常用词，也可能被切得更细；一个生僻词可能拆成好几块，一个标点或表情也可能单独占一块，甚至占据多块。实际切法由模型的分词器决定。因此，Token 数不等于字数，也不等于单词数。

大模型真正看到的，不是纸面上的整句话，而是这串编号小包裹及其前后关系。它根据已经收到的 Token，一步步生成后面的 Token，最后再由系统把这些小块还原成我们读到的回答。

当你向云端模型发送一段材料，材料、问题、必要的历史对话和系统指令，都可能形成输入 Token；模型生成回答，则形成输出 Token。不同服务的价格规则并不完全相同，但分别计算输入和输出 Token，是很常见的做法。

例如，用户只对 ASKS 说了一句：“把这篇论文整理进 Wiki。”看起来输入很短，但 Agent 真正交给大模型的，可能还包括论文正文、相关 Wiki 页面和任务规则。假设这些内容一共被切成 12,000 个输入 Token，模型生成整理方案用了 800 个输出 Token，那么这次调用记录的就是 12,000 个输入 Token 和 800 个输出 Token，而不只是用户说的那一句话。

如果检查后需要模型重新修改，还会产生新一轮 Token 用量；普通代码完成的格式检查则不消耗模型 Token。这里的 12,000 和 800 只是便于理解的示意数字，实际数量取决于材料长度、模型的分词器和真正送入模型的上下文。

这也解释了为什么两次看起来相近的任务，费用可能不同。一次只问一句话，另一次要附上几十页材料；一次直接得到结果，另一次需要反复修改。后者通常会让模型处理更多 Token。

但 Token 多，不代表答案一定更好。就像多用电不保证菜做得更香，Token 只说明模型处理了多少信息，不能替代任务设计、资料质量和结果检查。

大模型：为什么多数人选择“接入电厂”？

大模型的训练和运行需要大量计算资源。云端服务把这些资源集中起来，让个人和小团队通过 API 使用模型能力，而不必自己建设和维护整套基础设施。

这很像从公共电网取电：需要多少，用多少。家里也可以运行较小的本地模型，像使用家庭发电机或太阳能设备；而在本地部署非常大的模型，则更接近自建电厂，需要设备、能源和维护能力。

“自己不建电厂”不只是为了省事，首先是因为太贵。

这里需要区分闭源 GPT 服务和开放权重模型。对于 API 中的闭源 GPT，OpenAI 没有提供可下载的生产模型权重，普通用户并不存在“买几张显卡，在家运行同款满血 GPT”这条路；现实选择是通过 API 使用，并按照官方当期价格为 Token 付费。

开源模型虽然允许自己部署，也不等于普通电脑跑得动。以公开的满血 DeepSeek-R1 为例，它共有 6710 亿个参数，每次计算激活约 370 亿个参数。激活参数较少可以降低一次计算的工作量，却不意味着其他权重可以不装载。即使按 FP8、也就是每个参数大约占 1 字节，只把模型权重装入显存就接近 671GB；实际运行还要给缓存和程序留空间。比较现实的起点是 8 张 141GB 的 H200，或者更多张 80GB 级显卡。

按 2026 年 9 月常见市场价格做数量级估算：假设这类数据中心显卡每张约 20 万至 30 万元，8 至 16 张仅显卡就约为 160 万至 480 万元，整机、网络和存储还要另算；若按每卡每小时约 20 元至 30 元租用，全天运行一个月约为 12 万至 35 万元，还没有算额外的机房、电力、散热和运维。量化、降低并发或使用蒸馏小模型可以明显降低成本，但那已经不是同一条件下的“满血版”。这些数字只是把假设写明的粗略估算，不是采购报价，却足以解释为什么大多数个人和小团队选择按次调用，而不是拥有一座自己的“大模型电厂”。

不过，大模型本身仍主要负责接收信息并生成结果。它会解释、归纳、推理和写作，却不会因为“很聪明”就自动获得你电脑里的文件、浏览器、数据库和发布权限。

有了电厂，不等于已经有了洗衣机。



同一种模型能力，接入不同工具和流程，就能成为用途不同的 Agent

Agent：同一种能力，为什么能变成不同功能？

电器的差别，不只在于用了多少电，更在于内部结构不同。台灯把电变成光，洗衣机把电变成一套清洗流程。

Agent 也是如此。它把大模型接入工具、记忆、任务状态、权限和反馈流程，于是同一种模型能力可以被组织成完全不同的用途：有的 Agent 帮你查资料，有的修改代码，有的管理日程，有的维护知识库。

因此，Agent 不是“大模型换了一个名字”，也不只是“给大模型写一段更长的提示词”。

把两者分开看，是准确而且必要的：

- 大模型面对一次调用：读入一串 Token，根据上下文生成新的 Token，然后返回结果；
- Agent 面对一个目标：准备模型需要的信息，决定何时调用模型，使用哪些工具，观察执行结果，并判断任务是否还要继续。

同一个大模型可以装进不同 Agent，一个 Agent 也可以更换模型，甚至同时使用多个模型。这正说明它们属于不同层次。大模型提供理解和生成能力；Agent 把这种能力放入一个可以持续行动的系统中。

但“分开”不等于“互不相干”。多数智能 Agent 仍把大模型作为重要的判断部件，有些模型也能直接生成工具调用请求；只是文件是否真的被修改、工具是否获准运行、失败后是否重试，仍由模型外部的 Agent 系统控制。

对话助手像一盏智能台灯，复杂 Agent 更像自动化生产线：模型能力只是动力，工具和流程决定它最终能生产什么。

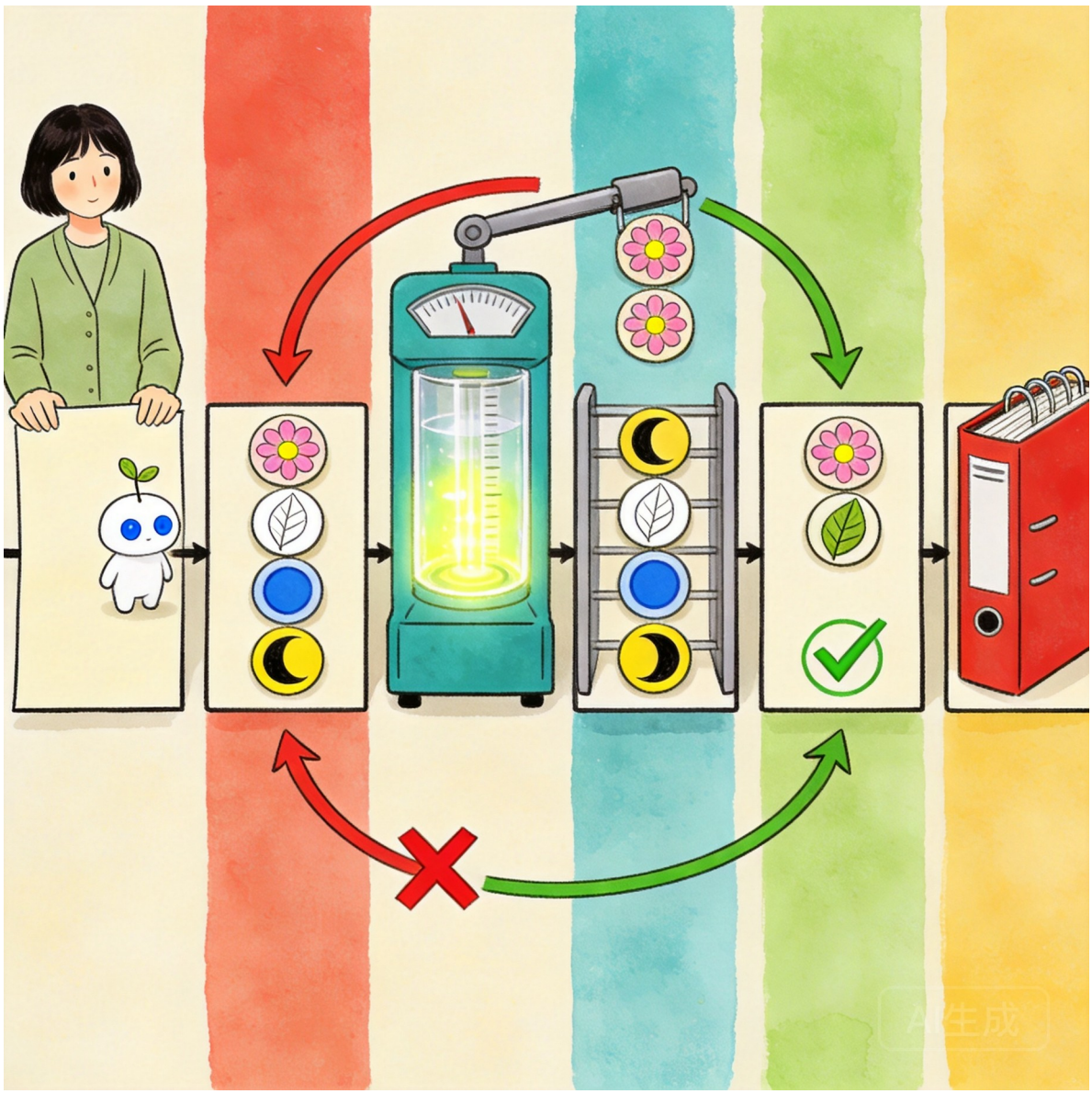
放进 ASKS 看一遍

假设研究者对 ASKS 说：“把这篇论文整理进 Wiki。”

首先，Agent 通过受管工具找到并读取论文。论文内容、已有 Wiki 中相关的信息和当前任务要求，会在需要模型判断时以 Token 的形式进入大模型。

大模型负责理解语义：论文研究了什么，哪些结论值得保留，新内容与已有认识是什么关系。它返回的分析同样由输出 Token 组成。

但事情没有停在一段回答。Agent 接着调用工具，把候选内容放到合适位置，并让确定性程序检查来源是否齐全、结构是否合规、同一材料是否被重复登记。如果检查失败，结果会返回给 Agent；Agent 再决定怎样修正，必要时重新调用模型。



在 ASKS 中，模型负责理解，Agent 组织行动，程序负责机械校验

这里的分工很关键：

- Token 是模型读入和生成信息时使用的基本单位，远程调用时也常决定费用；
- 大模型提供理解、推理和表达能力；

- Agent 持有任务的行动循环，组织模型、工具和检查程序共同完成目标。

读取本地文件、运行普通脚本，本身并不等于消耗模型 Token；只有信息被送入模型处理，才会产生相应的模型用量。机械校验也不能判断所有语义是否正确，它负责拦截规则明确的问题；真正的含义仍由模型结合上下文判断，重要结果还需要人确认。

别把比喻当成定义

“电厂—电力—电器”很好记，却不能完全对应现实。

电器消耗电力之后，电力就被用掉了；Agent 调用模型时，真正发生的是远程计算。Token 是被处理的信息单位和计量尺度，并不是装在电线里流动的物质。不同模型处理同样任务，效果、速度和价格也可能不同。

所以，这个比喻最值得保留的不是“AI 就是电”，而是三者之间的关系：

> Token 记录模型处理了多少信息，大模型提供通用语言能力，Agent 则把这种能力接入工具和流程，变成普通人可以直接使用的具体功能。

当我们为一个 Agent 付费时，常常是在为背后的模型调用付费；当我们评价一个 Agent 是否好用时，却不能只看它用了哪个模型，还要看它怎样组织工具、记忆、权限、校验和人的确认。

这就是从“大模型能回答什么”，走向“Agent 能替我们完成什么”的关键一步。

ASKS arXiv 论文

- Shi-Ju Ran, Kun Zhang, Xi Wu, Liu-Si Yang, and Wen-Jun Li. LLMs Interpret, Embeddings Organize, Graphs Emerge: Agent-Driven Compilation of Scientific Knowledge. arXiv preprint, arXiv:2608.29612 (2026). ASKS 的系统论文，介绍来源保留、持续知识编译和可重放更新。

相关论文推荐

- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1715-1725, Berlin, Germany (2016). 介绍把文本拆成子词单位的方法，有助于理解 Token 为什么不一定等于一个完整的词。
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint, arXiv:2210.03629 (2022; revised 2023). 展示语言模型怎样在推理、行动和观察之间循环，是理解大模型如何成为 Agent 一部分的经典工作。